

Uniswap UVN Core Audit



June 5, 2025

Table of Contents

Table of Contents	2
Summary	4
Scope	5
System Overview	7
L1 Contracts	7
L2 Contracts	9
Security Model and Trust Assumptions	10
Privileged Roles	11
Critical Severity	12
C-01 Slashing Calculation Excludes Withdrawn Amount	12
C-02 Malicious Operator Can Prevent reportOperatorStake from Executing Successfully on L2	12
High Severity	13
H-01 Rounding Errors In Slashing Lead to Announce Operator Undelegation Reverts	13
H-02 Incorrect Accounting Prevents Withdrawal	14
Medium Severity	14
M-01 Users Cannot Withdraw Funds When Delegate Is Not Set	14
M-02 The User's Rewards Are Not Correctly Slashed	15
M-03 Delegators Can Unfairly Boost UVN Reward Share by Delaying Slashing Application	15
Low Severity	16
L-01 Front-Running Configuring Delegator Access Control	16
L-02 Incorrect Naming for _delegatorAccessControl Mapping	17
L-03 Incorrect Order of Operations During Unstaking	17
L-04 Users May Be Unable to Withdraw UNI Tokens When Expected	17
L-05 Graffiti Can Be Overridden With the Same Signature	18
Notes & Additional Information	19
N-01 Ambiguous Calls to Parent Contract	19
N-02 Unnecessary Cast	19
N-03 Multiple Contract Declarations Per File	20
N-04 Lack of a Security Contact	20
N-05 Lack of Indexed Event Parameters	21
N-06 Unaddressed "TODO" Comments	21
N-07 Hardcoded Address	22
N-08 The Contracts May Not Be Compatible With ERC-4337	22
N-09 Use calldata Instead of memory	23

N-10 Floating Pragma	23
N-11 Possible Duplicate Event Emissions	24
N-12 Different Pragma Directives	24
N-13 Missing Named Parameters in Mappings	25
N-14 Missing Or Incomplete Docstrings	25
Conclusion	27

Summary

Type	DeFi	Total Issues	26 (18 resolved)
Timeline	From 2025-04-22 To 2025-05-28	Critical Severity Issues	2 (2 resolved)
Languages	Solidity	High Severity Issues	2 (2 resolved)
		Medium Severity Issues	3 (2 resolved)
		Low Severity Issues	5 (3 resolved)
		Notes & Additional Information	14 (9 resolved)

Scope

OpenZeppelin audited the [Uniswap/unichain-contracts/](https://github.com/Uniswap/unichain-contracts/) repository at commit [d7e11d](https://github.com/Uniswap/unichain-contracts/commit/d7e11d).

In scope were the following files:

```
src
├── UVN
│   ├── L1
│   │   ├── StakeTableSync.sol
│   │   ├── StakingMiddleware.sol
│   │   └── StakingMiddleware
│   │       ├── DelegatorAccessControl.sol
│   │       ├── Notifier.sol
│   │       ├── OperatorManager.sol
│   │       ├── ProtocolRewardDistributor.sol
│   │       ├── SlashingManager.sol
│   │       ├── StakeManager.sol
│   │       ├── StakingMiddlewareParams.sol
│   │       ├── UniStakerWrapper.sol
│   │       └── libraries
│   │           ├── OperatorTokenLib.sol
│   │           ├── OperatorVotes.sol
│   │           └── Votes.sol
│   └── L2
│       ├── DefaultDelegatorClaim.sol
│       ├── L2StakeTable.sol
│       ├── RewardDistributor.sol
│       ├── RewardDistributorParams.sol
│       └── libraries
│           ├── AttestationLib.sol
│           ├── Search.sol
│           └── WindowLib.sol
└── interfaces
    ├── UVN
    │   ├── IBaseService.sol
    │   ├── L1
    │   │   ├── IDelegationManager.sol
    │   │   ├── IService.sol
    │   │   ├── IStakeTableSync.sol
    │   │   ├── IStakingMiddleware.sol
    │   │   ├── IUniStaker.sol
    │   │   └── StakingMiddleware
    │   │       ├── IDelegatorAccessControl.sol
    │   │       ├── IDelegatorVerifier.sol
    │   │       ├── INotifier.sol
    │   │       ├── IOperatorManager.sol
    │   │       ├── IProtocolRewardDistributor.sol
    │   │       └── ISlashingManager.sol
```

```
|      |— IStakeManager.sol
|      |— IStakingMiddlewareParams.sol
|      |— IUniStakerWrapper.sol
|— L2
|   |— IDefaultDelegatorClaim.sol
|   |— IDelegatorClaim.sol
|   |— IL2StakeTable.sol
|   |— IOperatorFeeManager.sol
|   |— IRewardDistributor.sol
|   |— IRewardDistributorParams.sol
|   |— IRewardPuller.sol
|   |— IStakeTable.sol
```

System Overview

The UVN staking system allows users to delegate tokens to operators who attest to windows of blocks. Delegators earn rewards for their operator's correct attestations but also bear the risk of slashing if their chosen operator behaves maliciously. The system is divided into L1 (Layer 1) and L2 (Layer 2) components. The L1 contracts handle core staking operations such as staking, unstaking, and withdrawing, as well as voting and slashing. The L2 contracts manage attestation processes and reward distribution.

L1 Contracts

The core L1 contract is `StakingMiddleware`, which inherits multiple modules, each responsible for specific functionality. These modules frequently use hooks to execute additional logic before or after the main function logic. In addition, the `StakeTableSync` contract has been implemented to sync stake data from the `StakingMiddleware` contract to the `L2StakeTable` contract. It forwards updates on slashing, balance changes, and operator token deposits to ensure accurate stake balances on L2. It also provides functions to manually correct any stake inconsistencies for delegators. The `StakingMiddleware` utilizes the following modules:

`DelegatorAccessControl`

This contract manages delegation access control by allowing operators to define the conditions under which delegators can delegate to them. Self-delegation is always permitted, ensuring that users can delegate to themselves without restriction. Operators have the ability to toggle whether delegation to them is allowed. When delegation is enabled, operators can enforce custom access policies through one of two mechanisms.

The first mechanism allows an operator to specify a verifier contract that contains logic to determine whether a given delegator is authorized to delegate to the operator. The second mechanism allows the operator to define an `authorizedSender` address, typically a contract, that is permitted to submit delegation signatures.

Notifier

The L1 contracts communicate state changes to the service contracts via the `Notifier` contract. This allows operators to mint an NFT with a token ID equivalent to their contract address. This NFT can be deposited into service contracts which will be notified when slashes occur or the stake is changed. The Notifier makes external calls to these service contracts, allowing them to implement custom logic during deposits, withdrawals, and slashing operations.

OperatorManager

The contract governs how delegators select operators, implementing the `IVotes` interface to manage delegation relationships. To undelegate from an operator, a delegator must first undergo a delay period. During this period, the delegator's voting power is reduced to zero, but they remain subject to slashing penalties until the undelegation process is finalized.

ProtocolRewardDistributor

This contract is responsible for distributing accrued protocol fees to delegators who have opted to deposit their underlying `UNI` tokens stake into the `UniStaker` contract.

SlashingManager

When slashing is applied to an operator, the stake of all associated delegators is reduced by the same percentage. The `SLASHER_ROLE` can either specify an amount or a percentage to slash. Rewards accrued by the slashed stake are also reduced and redirected to the slashing beneficiary.

When a state-changing function is called in the `StakingMiddleware` contract, the `applySlashing` function is called through a hook to ensure that slashing is applied before a delegator's stake is changed. Slashing reduces voting power and delegator stakes by the same percentage.

StakeManager

This contract has been designed to manage the staking activity of a delegator. It allows delegators to stake `UNI` tokens and initiate a withdrawal after a specified delay period. Once a delegator requests to unstake their tokens, the stake remains subject to slashing until the withdrawal is fully completed. This means that even if the withdrawal delay has passed, the stake can still be slashed if it has not yet been withdrawn. Slashing is applied proportionally to the entire stake, including any pending withdrawals.

StakingMiddlewareParams

This contract is responsible for managing the configuration parameters of the `StakingMiddleware` contract. It provides authorized roles with the ability to set the withdrawal delay and designate the slashing beneficiary.

UniStakerWrapper

This contract is responsible for managing deposits into the `UniStaker` contract. It enables delegators to take part in UNI governance and to earn protocol fees that are distributed by the `UniStaker` contract. Depositing into the `UniStaker` contract is optional. However, once a delegator chooses to opt in, all future deposits will be automatically directed to the `UniStaker` contract.

L2 Contracts

Delegator balances on L2 are synchronized via L1 messages sent to the `L2StakeTable` contract through the `reportOperatorStake` and `reportOperatorSlash` functions. Each operator's attestations are stored in a linked list structure, where each node records the voted hash and the corresponding number of votes.

Rewards are distributed proportionally among operators who had attested to the hash that received the most votes within an attestation window. A window is considered successfully finalized if a single hash receives attestations exceeding two-thirds of the total voting supply. If no hash meets this threshold, the attestation result for that window is marked as `InsufficientVotes`.

L2StakeTable

This contract has been designed to synchronize with L1 such that any changes to an operator's balance on L1 trigger a notification to this contract, prompting an update to the corresponding balance on L2. Following this deployment, the contract continues to receive notifications of any subsequent stake changes occurring on L1. In addition, operators have the ability to override the default delegator claim contract by providing a custom implementation, enabling them to distribute rewards according to their own logic.

DefaultDelegatorClaim

This contract is responsible for distributing rewards that it receives from the reward distributor contract. Moreover, it allows an operator to implement custom logic to manage how their fees are handled.

RewardDistributor

This contract manages the distribution of rewards to operators based on their attestations. Operators submit attestations for groups of blocks, referred to as "windows". When a window is finalized, rewards are distributed to the operators who voted with the majority within that window. To ensure that rewards are allocated correctly, if no attestations have been made for a scheduled window, the activation of that window is delayed by extending it to the previous block before it becomes active.

RewardDistributorParams

It is a contract that `RewardDistributor` inherits from, and it implements the logic for setting reward distributor parameters such as the attestation window length, attestation period, and reward puller address.

Security Model and Trust Assumptions

The following trust assumptions were made as part of the audit:

- The UVN contracts depend on external service contracts which are outside the scope of this audit. The security of the system partly relies on the correctness and security of these external contracts.
- The delegator's reward amount on L2 is only reduced after slashing is applied on L1. Delegators to the same operator can have their L2 balances updated at different times which can lead to discrepancies in rewards.
- The L1 contracts are immutable and not upgradeable post-deployment whereas the L2 contracts are.
- Potential discrepancies may arise between the states tracked on L1 and L2, particularly concerning slashing operations and reward distribution. Slashing initiated on L1 may not immediately reflect on L2, creating windows where reward distribution could become temporarily inconsistent or unfair. Ensuring consistency between layers requires reliable message passing via the Optimism Portal.

- Rounding errors can lead to minor differences between the total stakes of delegators and the total voting power operators. These rounding errors are small but may accumulate over time.

Privileged Roles

The project implements the following privileged roles:

- On L1, `DEFAULT_ADMIN_ROLE` manages role assignments and critical admin operations.
- On L1, `SLASHER_ROLE` can call functions to slash operator and delegator stakes.
- On L1, `TRUSTED_SERVICE_ROLE` allows service contracts to revert withdrawals and slashing operations.
- On L2, `PARAM_SETTER_ROLE` can change the attestation period, attestation window length, and the reward puller address.

Critical Severity

C-01 Slashing Calculation Excludes Withdrawn Amount

In the `withdraw` function, the `withdrawn` amount is immediately subtracted from `stake_.totalPendingWithdrawal` before calling the `_beforeWithdraw()` hook. The slashing logic in the `_beforeWithdraw()` hook applies slashing based on `totalPendingWithdrawals`. However, since the withdrawal amount has already been deducted, the slashing logic incorrectly excludes the withdrawn amount. In addition to incorrect slashing calculation, this can also cause the subtraction of `slashableStakes` in the `_afterWithdraw` hook to underflow which would prevent withdrawals.

Consider slashing the user's balance before subtracting the withdrawn amount from it.

Update: Resolved in [pull request #64](#). The Uniswap team stated:

The slashing is now applied before assets are withdrawn (this will essentially brick withdrawals though as they will be rescheduled, but now everything should be correct).

C-02 Malicious Operator Can Prevent `reportOperatorStake` from Executing Successfully on L2

The `reportOperatorStake` function of the `L2StakeTable` contract is responsible for updating an operator's voting units and notifying delegator claim contracts when there are changes in delegator stakes. However, there is a potential issue with the external call to the `delegatorClaim` contract, which is currently wrapped in a `try-catch` block. While this approach suggests that errors are being handled, it does not guarantee that the transaction will not revert.

In Solidity, a `try-catch` block can still lead to a revert if the call is made to an address that contains no contract code. This situation can occur if a user invokes the `overrideDelegatorClaimContract` function and sets the delegator claim contract address to an address with no deployed code.

Consider wrapping the call to `delegatorClaim` in an additional `try-catch` block that would successfully handle any reverts caused by calls to addresses that do not contain contract code. Alternatively, consider adding a check before making the external call to verify that the target address contains contract code.

Update: Resolved in [pull request #75](#).

High Severity

H-01 Rounding Errors In Slashing Lead to Announce Operator Undelegation Reverts

Multiple instances of rounding errors that lead to incorrect accounting and reverts were identified in `announceOperatorUndelegation`. When the operator is slashed twice, the slashed amount is rounded down. However, during the calculation in `announceOperatorUndelegation`, the calculated slash value is one unit smaller than the actual amount the operator was slashed for.

The following scenario demonstrates the calls that lead to incorrect accounting:

1. The operator mints a token.
2. The operator sets the delegation status to `true`.
3. The user delegates to the operator.
4. The user stakes 9 tokens.
5. The operator gets slashed for 1 token.
6. The operator gets slashed for 1 token again.
7. The `announceOperatorUndelegation` call keeps reverting in the `OperatorVotes` contract because it attempts to subtract the account's voting units 6 from the total checkpoint value 5, which results in an underflow.

Consider redesigning the logic to account for rounding when multiple slashes are applied to a user's stake.

Update: Resolved in [pull request #64](#).

H-02 Incorrect Accounting Prevents Withdrawal

The withdrawal logic implemented in the `withdraw` function of `StakeManager` calculates the total amount of all pending withdrawals, which are then withdrawn. The issue is that this amount is passed to the `__beforeWithdraw` hook, which is implemented in `UniStakerWrapper` and attempts to `withdraw` the same amount from `UniStaker`. The problem with this approach is that not all tokens pending withdrawal were necessarily deposited into `UniStaker`. A user might stake tokens and then unstake a portion of them without ever depositing into `UniStaker`. If the user later deposits into `UniStaker`, only the currently staked tokens are considered, while pending withdrawals were never deposited there. This means it's not possible to withdraw tokens, as the transaction keeps reverting due to attempting to withdraw more from the `UniStaker` deposit than was originally deposited.

The following scenario presents the calls that lead to incorrect accounting: 1. Alice stakes 10 tokens. 2. Alice unstakes 9 tokens. 3. Alice deposits into `UniStaker` via `depositIntoUniStaker`, creating a deposit with a stake size of 1 token. 4. Alice attempts to withdraw, but the transaction keeps reverting because it tries to withdraw from `UniStaker` the total amount equal to the sum of all pending withdrawals which is 9 while the deposit in `UniStaker` holds only 1 token.

Consider accurately tracking the stake amount to ensure that only the slashable portion is taken into account.

Update: Resolved in [pull request #60](#).

Medium Severity

M-01 Users Cannot Withdraw Funds When Delegate Is Not Set

Once a user stakes funds, the logic in the `__afterStake` hook is triggered. If an operator is set, the `__slashableStakes` value is increased accordingly. The issue arises when the user withdraws funds, triggering the `__afterWithdraw` hook, which attempts to reduce the `__slashableStakes`. If the user does not have a delegate set, this `__afterWithdraw` hook attempts to subtract the `amount` from `__slashableStakes` of the zero address, which results in overflow.

Consider adding a check that only reduces the slashable stake of the operator if the operator is set.

Update: Resolved in [pull request #61](#). The Uniswap team stated:

| Fixed by adding the missing operator check.

M-02 The User's Rewards Are Not Correctly Slashed

The `applySlashing` function does not call `_distributeRewards` if the reward amount is 0. Therefore, it does not update the reward checkpoint of the user to the global checkpoint. A reward amount of 0 can occur if 100% of a user's stake is slashed. In this case, after `applySlashing` is called, rewards are again distributed as the `rewardCheckpoint` of the user is not equal to the `globalCheckpoint`, but this time without the slashing applied. Therefore, the user can be credited the rewards that should have been slashed.

Consider always updating the global reward checkpoint to ensure accurate reward calculations.

Update: Resolved in [pull request #67](#).

M-03 Delegators Can Unfairly Boost UVN Reward Share by Delaying Slashing Application

When slashing occurs, a delegator's stake on L2 is not reduced until `applySlashing` is called on L1. This can lead to inaccurate L2 reward distribution when an operator has multiple delegators and some delegators have had slashing applied while others have not. Calling `applySlashing` reduces both the delegator's tracked stake and their accrued Unistaker rewards, but does not affect operator rewards. Delegators can call `claimRewards` on L2 which does not enforce the stake reduction from slashing. Since L2 rewards are distributed proportionally based on each delegator's stake (`delegationOf[account]`) relative to `totalDelegation_`, this behavior results in an unfair distribution of rewards.

Delegators who delay applying slashing retain an inflated `delegationOf[account]` value, benefiting from an artificially higher share of rewards as the `totalDelegation` is reduced by other delegators who already have had slashing applied. Conversely, delegators who apply slashing promptly receive fewer rewards than their fair share. A delegator can call `applySlashing` on other delegators but not themselves in order to reduce others' reward

share and, therefore, claim more rewards themselves. This reward discrepancy can be fixed manually by a user calling `applySlashing` for all delegators.

On L2, consider slashing operators and their respective delegators simultaneously.

Update: Acknowledged, not resolved. The Uniswap team stated:

The main issue with applying slashing to delegators is that we would need to not only apply the slashing but also adjust the checkpoints to take the new stake into account. As this would be an $O(n)$ operation where n is the number of delegators, this could potentially run out of gas. Because of the try catch call this could potentially lead to incorrect values after slashing. Therefore we opted into the approach of applying the slashing individually, to ensure that the balances of delegators remain accurate on the L2. So this issue is the intended behavior, as other delegators have the incentive to apply slashing for delegators whose balances have not been slashed on L2, yet.

Low Severity

L-01 Front-Running Configuring Delegator Access Control

The `DelegatorAccessControl` contract allows for configuring `AccessControlParams`, but this can only be done through [three separate external calls](#), each of which sets a specific part of the configuration. This introduces a risk of front-running by delegators, who may delegate to the operator before the configuration is fully and correctly set.

Consider either thoroughly documenting this behavior so that operators are aware of the correct order, or ensuring that the configuration is applied atomically, ideally within a single transaction.

Update: Acknowledged, not resolved. The Uniswap team stated:

The contract inherits `multicall`, if an operator wants to prevent frontrunning, they can use the `multicall` to execute all updates at once. Additionally, disabling delegation allows to set the other parameters without any risk of frontrunning, allowing to enable the delegation afterwards without any risk.

L-02 Incorrect Naming for `_delegatorAccessControl` Mapping

The `_delegatorAccessControl` mapping is labelled as being a mapping from `delegator` to `accessControl`. However, it actually maps `operator` to `accessControl`.

Consider renaming the `delegator` mapping to `operator`.

Update: Resolved in [pull request #66](#).

L-03 Incorrect Order of Operations During Unstaking

The `unstake` function of the `StakeManager` contract triggers the `_beforeUnstake` hook, which may apply slashing. However, the current stake is `cached before` the slashing occurs, making the `subsequent check` for whether the unstake amount exceeds the stake unreliable. This can lead to an overflow when subtracting the unstake amount from the potentially reduced stake.

Consider triggering the `_beforeUnstake` hook before caching, or checking the user's stake.

Update: Resolved in [pull request #62](#).

L-04 Users May Be Unable to Withdraw UNI Tokens When Expected

The logic for unstaking and withdrawing UNI tokens, as implemented in the `StakeManager` contract, relies on the assumption that pending withdrawals are ordered by timestamp. If the next withdrawal is scheduled for the future, [the process stops](#) and does not proceed with subsequent pending withdrawals. While this behavior is generally correct, there is a case where pending withdrawals might not be ordered by timestamp - specifically, when `_withdrawalDelay` in `StakingMiddlewareParams` is [changed to a value that is smaller than the current one](#).

The following scenario illustrates the issue:

1. `withdrawalDelay` is initially set to 100.

2. The user unstakes at `block.timestamp` = 100, which sets the withdrawal availability to `timestamp` = 200.
3. The admin updates `withdrawalDelay` to 50.
4. The user unstakes again at `block.timestamp` = 110, setting the withdrawal availability to `timestamp` = 160.
5. The pending withdrawals now include entries with the following availability timestamps: 200 and 160.
6. The user cannot withdraw any funds until `block.timestamp` reaches 200, even though the second withdrawal becomes available at `block.timestamp` = 160.

Consider either thoroughly documenting this behavior, or adjusting the logic to dynamically calculate the undelegation time based on the current withdrawal delay during the withdrawal process. However, note that adjusting the logic could create an issue where the user is uncertain about when their funds will be available because any change in the withdrawal delay would affect all pending withdrawals.

Update: Resolved in [pull request #63](#). The Uniswap team stated:

The withdrawal timestamp is now calculated dynamically. Generally we are not expecting this parameter to be updated often, if ever.

L-05 Graffiti Can Be Overridden With the Same Signature

The `attest` function can be successfully called by any user as long as the operator's address can be recovered from the signature. If an operator calls `attest`, a frontrunner can submit a call to attest with the same `blockNumber`, `blockHash`, `additionalData`, and `signature` parameters but with a different `graffiti` value. While the operator's vote remains the same, the `graffiti` can be altered. The `graffiti` allows operators to include information about their node in the attestation, and changing it can lead to misleading event emissions.

Consider implementing one of the following possible remediations:

- Include `graffiti` in the signed data. This way, only the operator can set the graffiti.
- Remove the `graffiti` variable entirely if it does not serve a useful purpose.
- Document the potential frontrunning of the `graffiti` value so that operators are aware of this possibility.

Update: Acknowledged, not resolved. The Uniswap team stated:

The graffiti is a feature similar to the beacon chain graffiti where validators can pass in arbitrary data that is not useful in the consensus process, therefore the graffiti is also not included in the signature. However, initially we intend to use it to relay metadata about the node (e.g., software version) to see what percentage of validators has updated their software to the latest version for example. Additionally, front-running of the attestation is highly unlikely as Unichain uses a private mempool and thus the transaction has to leak in some way to make it front-runnable. So, in the Unichain specific context, it should not be a real issue. Should this become an issue in the future for whatever reason, we could redeploy the reward distributor and require the sender of the transaction to be included in the additional data.

Notes & Additional Information

N-01 Ambiguous Calls to Parent Contract

Throughout the codebase, multiple instances of ambiguous calls to parent contract were identified:

- `super.supportsInterface` in the `Notifier` contract
- `super.rewardsOf` in the `SlashingManager` contract
- `super.supportsInterface` in the `StakeTableSync` contract
- `super.nonces` in the `StakingMiddleware` contract

Consider explicitly specifying the parent contracts whose functions are being called to improve the clarity and maintainability of the codebase.

Update: Resolved in [pull request #68](#). The Uniswap team stated:

Used external calls except for ERC165 supportsInterface calls. Using super here ensures that all base contracts are called even when adding more modules that might also use ERC165

N-02 Unnecessary Cast

Within the `StakeManager` contract, the `uint96(newStake)` cast is unnecessary.

To improve the overall clarity and maintainability of the codebase, consider removing any unnecessary casts.

Update: Resolved in [pull request #64](#). The Uniswap team stated:

| *Fixed accidentally in the following PR*

N-03 Multiple Contract Declarations Per File

Within [WindowLib.sol](#), there are multiple contract or library declarations.

Consider separating the contracts into their own files to make the codebase easier to understand for developers and reviewers.

Update: Resolved in [pull request #74](#).

N-04 Lack of a Security Contact

Providing a specific security contact (such as an email or ENS name) within a smart contract significantly simplifies the process for individuals to communicate if they identify a vulnerability in the code. This practice is quite beneficial as it permits the code owners to dictate the communication channel for vulnerability disclosure, eliminating the risk of miscommunication or failure to report due to a lack of knowledge on how to do so. In addition, if the contract incorporates third-party libraries and a bug surfaces in those, it becomes easier for their maintainers to contact the appropriate person about the problem and provide mitigation instructions.

Consider adding a NatSpec comment containing a security contact above each contract definition. Using the [@custom:security-contact](#) convention is recommended as it has been adopted by the [OpenZeppelin Wizard](#) and the [ethereum-lists](#).

Update: Resolved in [pull request #69](#). The Uniswap team stated:

| *Added suggested natspec to high level contracts and added security markdown file*

N-05 Lack of Indexed Event Parameters

Throughout the codebase, multiple instances of events not having indexed parameters were identified:

- The `RewardDistributed` event in `IDefaultDelegatorClaim.sol`
- The `OperatorFeeManagerSet` event in `IDefaultDelegatorClaim.sol`
- The `RewardsAdded` event in `IProtocolRewardDistributor.sol`
- The `AttestationWindowExtended` event in `IRewardDistributor.sol`
- The `AttestationWindowActivated` event in `IRewardDistributor.sol`
- The `AttestationWindowLengthUpdated` event in `IRewardDistributorParams.sol`
- The `AttestationPeriodUpdated` event in `IRewardDistributorParams.sol`
- The `RewardPullerUpdated` event in `IRewardDistributorParams.sol`
- The `WithdrawalDelayUpdated` event in `IStakingMiddlewareParams.sol`
- The `SlashingBeneficiaryUpdated` event in `IStakingMiddlewareParams.sol`
- The `RewardNotified` event in `IUniStaker.sol`

To improve the ability of off-chain services to search and filter for specific events, consider [indexing event parameters](#).

Update: Resolved in [pull request #70](#). The Uniswap team stated:

Added indices where it made sense. Notes: - IUniStaker.sol is an interface for the implementation of the UniStaker contract developed by Uniswap Foundation (because that contract didn't have an interface) and thus I can't change it. - My understanding of indexing is that it's mainly used to filter topics for specific arguments. So I think it doesn't really make sense to index amounts or block numbers that will most likely only show up once or infrequently, similarly how the amount is not indexed in ERC20 tokens. Please correct me if I'm wrong

N-06 Unaddressed "TODO" Comments

During development, having well-described TODO/Fixme comments will make the process of tracking and solving them easier. However, without being addressed timely, these comments might age and important information for the security of the system might be forgotten by the time it is released to production. As such, these comments should be tracked in the project's issue backlog and resolved before the system is deployed.

There is a `TODO` comment in [line 116 of UniStakerWrapper.sol](#).

Consider removing all instances of TODO/Fixme comments and instead tracking them in the issues backlog. Alternatively, consider linking each inline TODO/Fixme to the corresponding issues backlog entry.

Update: Resolved in [pull request #71](#). The Uniswap team stated:

Oops I was sure I deleted it, might have snuck back during a merge. Looking at the comment it should not be necessary to revert anymore like the TODO comment suggests. Can you confirm?

N-07 Hardcoded Address

The `0x0bd48f6B86a26D3a217d0Fa6FfE2B491B956A7a2` value within `StakeTableSync.sol` is a hardcoded address.

Consider declaring hardcoded addresses as `immutable` values and assigning them via constructor arguments. This allows the code to remain the same across deployments on different networks, and avoids error-prone address hardcoding and recompilation when addresses need to change.

Update: Acknowledged, not resolved. The Uniswap team stated:

Acknowledged, this will only be deployed on Ethereum and it points to the OptimismPortal2 contract of Unichain, so imo it's fine to hard code

N-08 The Contracts May Not Be Compatible With ERC-4337

EIP-4337 is a technique that allows smart contracts to behave like user accounts. Under this EIP, there will be EIP-4337 smart contract accounts in addition to Externally Owned Accounts (EOAs). The former account type is incompatible with some code, such as `tx.origin` or `ecrecover`.

Throughout the codebase, multiple instances of contracts that may not be compatible with ERC-4337 were identified:

- The `RewardDistributor` contract in `RewardDistributor.sol`
- The `Votes` contract in `Votes.sol`

To prevent incompatibility with EIP-4337 accounts, consider using `msg.sender` instead of `tx.origin`, and using [EIP-1271](#) to allow smart contract accounts to verify signatures instead of `ecrecover`. In addition, consider using the [SignatureChecker](#) library instead of [ECDSA](#).

Update: Acknowledged, not resolved. The Uniswap team stated:

| *Acknowledged, operators are intended to be EOAs*

N-09 Use `calldata` Instead of `memory`

When dealing with the parameters of `external` functions, it is more gas-efficient to read their arguments directly from `calldata` instead of storing them to `memory`. `calldata` is a read-only region of memory that contains the arguments of incoming `external` function calls. This makes using `calldata` as the data location for such parameters cheaper and more efficient compared to `memory`. Thus, using `calldata` in such situations will generally save gas and improve the performance of a smart contract.

Throughout the codebase, multiple instances where function parameters should use `calldata` instead of `memory` were identified:

- In `Notifier.sol`, the `uri` parameter
- In `RewardDistributor.sol`, the `additionalData` parameter
- In `RewardDistributor.sol`, the `signature` parameter

Consider using `calldata` as the data location for the parameters of `external` functions to optimize gas usage.

Update: Resolved in [pull request #72](#).

N-10 Floating Pragma

Pragma directives should be fixed to clearly identify the Solidity version with which the contracts will be compiled.

Throughout the codebase, multiple instances of floating pragma directives (such as `^0.8.0` and `^0.8.26`) were identified.

Consider using fixed pragma directives.

Update: Acknowledged, will resolve in [pull request #73](#). The Uniswap team stated:

N-11 Possible Duplicate Event Emissions

Throughout the codebase, multiple instances where an event could be emitted despite the value remaining unchanged were identified.

- `_setAttestationPeriod` sets the `_attestationPeriod` and emits an event without checking if the value has changed.
- `_setWithdrawalDelay` sets the `_withdrawalDelay` and emits an event without checking if the value has changed.
- `_setSlashingBeneficiary` sets the `_slashingBeneficiary` and emits an event without checking if the value has changed.

Consider adding a check that reverts the transaction if the value being set is identical to the existing value.

Update: Acknowledged, not resolved. The Uniswap team stated:

Acknowledged

N-12 Different Pragma Directives

In order to clearly identify the Solidity version with which the contracts will be compiled, pragma directives should be fixed and consistent across file imports.

Throughout the codebase, different Solidity versions are being used including `^0.8.0`, `0.8.26`, and `^0.8.20`.

Consider using the same, fixed pragma version in all files.

Update: Acknowledged, will resolve. The Uniswap team stated:

- 0.8.26 was used for all contracts (except for `Notifier.sol`, will fix that). - ^0.8.20 was used for the `Votes` contract which was copied directly from `openzeppelin`. To ensure minimal changes to the contract I would prefer to not change the pragma. - All interfaces use `^0.8.0` to ensure that developers integrating with the contracts are not limited to developing their contracts with 0.8.26. Using `^0.8.0` for the contracts ensures a better development experience.

N-13 Missing Named Parameters in Mappings

Since [Solidity 0.8.18](#), developers can utilize named parameters in mappings. This means mappings can take the form of `mapping(KeyType KeyName? => ValueType ValueName?)`. This updated syntax provides a more transparent representation of a mapping's purpose.

Within `Votes.sol`, the `_delegatee` state variable and the `_delegateCheckpoints` state variable do not have named parameters.

Consider adding named parameters to mappings in order to improve the readability and maintainability of the codebase.

Update: Resolved. The Uniswap team stated:

`_Votes` was copied from `openzeppelin`, I would like to keep changes to an absolute minimum, therefore I'd prefer to not change anything about this contract that isn't necessary _

N-14 Missing Or Incomplete Docstrings

Throughout the codebase, multiple instances of missing docstrings were identified:

Contracts and Libraries:

- `DefaultDelegatorClaim` contract
- `RewardDistributorParams` contract
- `OperatorTokenLib` library
- `Search` library

Interfaces:

- `IDelegationManager` interface and its functions
- `IDelegatorClaim` interface and the `reportDelegatorStake` function
- `IRewardDistributor` interface
- `IRewardDistributorParams` interface
- `IRewardPuller` interface
- `IStakeTable` interface
- `IUniStaker` interface

Functions:

- In `Notifier.sol`, the `tokenURI` and `supportsInterface` functions
- In `StakingMiddlewareParams.sol` the `withdrawalDelay`, `slashingBeneficiary`, `updateWithdrawalDelay` and `updateSlashingBeneficiary` functions

Consider thoroughly documenting all functions (and their parameters) that are part of any contract's public API. Functions implementing sensitive functionality, even if not public, should be clearly documented as well. When writing docstrings, consider following the [Ethereum Natural Specification Format](#) (NatSpec).

Update: Resolved in [pull request #76](#).

Conclusion

The audited UVN contracts enable delegators to stake **UNI** tokens and delegate them to operators who perform attestations for block windows. Operators acting maliciously can cause their delegators' stakes to be slashed, while operators who successfully attest to blocks earn rewards that are distributed among their delegators.

Multiple critical-, high-, and medium-severity vulnerabilities were identified during the audit, while code improvement recommendations were also made. Given the state-based logic governing staking, unstaking, and slashing, it is highly recommended to employ fuzzing and formal verification techniques to ensure smooth and secure state transitions.

The Uniswap team is appreciated for providing comprehensive audit tags within the codebase, highlighting system invariants and essential context for the audit. They also presented a detailed overview of the protocol, promptly addressed auditor questions, and implemented the recommended fixes.